

打刻内容編集申請機能の実装

打刻漏れ（出勤時の押し忘れ、退勤時の押し間違い）や、急な直行・直帰による時刻修正など、過去の打刻データを修正・申請する機能は、ユーザーが日々の運用で**最も頻繁に、かつ速やかな対応を必要とする最重要機能**の一つです。

これを「予定申請」より先に実装することは、アプリケーションの価値を実用面で一気に高めることに繋がります。

「打刻内容編集申請」の仕様と設計の考察

今回の要件は以下の通りです。

- 操作日の当月1か月分のリストカレンダーを表示する
- リストのボタン（行）を押下することで編集フォームを表示する
- Entry（出勤/入室等）/ Exit（退勤/退室等）時刻を `hh:mm:ss` 形式で修正できる
- 備考（修正理由など）の入力欄を設ける
- 申請時は必ず確認アラートを表示する
- データベース（`entry_exit_calendar` テーブル）の状態管理
 - `is_timechange_demand` : 1（時刻修正申請中であることを示すフラグ）
 - `timechange_status` : 0（未承認）、1（差戻）、2（承認済）

データの流れと安全策（2重バリデーション）の設計

ユーザーが過去の日付を選んで「Entry 09:00:00 / Exit 18:00:00」と入力して申請ボタンを押した際、システムの的に以下の考慮が必要になります。

- **フロントエンド（Vue.js）での制御：**
 - 入力された文字列が正しい時刻形式（`hh:mm:ss`）になっているかバリデーションを実施します。
 - 備考欄が空欄の場合、「修正理由を入力してください」と警告を出します（不正防止のため必須化が望ましいです）。
- **バックエンド（SpringBoot）での制御：**
 - 送られてきた `accountId` と、ログイン中のセッションユーザーが一致しているかチェックします（他人の打刻を勝手に書き換える不正を防止）。
 - 該当日のレコードが `entry_exit_calendar` に存在すれば「アップデート（申請中フラグと修正案のセット）」、まだ打刻レコード自体が存在しない日（丸ごと押し忘れた日）であれば「新規に申請中レコードをINSERT」という2パターンの制御を優しいJPAロジックで捌きます。

バックエンド（Java）の実装

まずは、ユーザーからの「この日の打刻をこう直したいです」という申請リクエストを受け付けるAPIを構築しましょう。

既存の `entry_exit_calendar` テーブル（対応する Entity）には、修正内容を一時保存できる「申請中 Entry時刻」「申請中Exit時刻」「申請メッセージ（備考）」用のカラム（プロパティ）が必要になります。

リクエストを受け取る DTO クラスの作成

ユーザーからの申請データをスマートに格納する器（`TimeChangeRequest.java`）を作成します。

（`backend` 内に `dto` フォルダを作成していない場合は、格納に必要なので予め作成しておいて下さい）

```
package com.appspace.backend.dto;

import java.time.LocalDate;

public class TimeChangeRequest {
```

```
private String accountId;
private LocalDate recordDate; // 修正対象の日付 (yyyy-MM-dd)
private String targetEntryTime; // 修正希望Entry時刻 (hh:mm:ss)
private String targetExitTime; // 修正希望Exit時刻 (hh:mm:ss)
private String reason; // 編集申請の備考・理由
// ゲッター、セッター
public String getAccountId() { return accountId; }
public void setAccountId(String accountId) { this.accountId = accountId; }

public LocalDate getRecordDate() { return recordDate; }
public void setRecordDate(LocalDate recordDate) { this.recordDate = recordDate; }

public String getTargetEntryTime() { return targetEntryTime; }
public void setTargetEntryTime(String targetEntryTime) { this.targetEntryTime = targetEntryTime; }

public String getTargetExitTime() { return targetExitTime; }
public void setTargetExitTime(String targetExitTime) { this.targetExitTime = targetExitTime; }

public String getReason() { return reason; }
public void setReason(String reason) { this.reason = reason; }
}
```

コントローラーへのエンドポイント追加

既存の `AttendanceController.java`、または新設の `AttendanceEditController.java` に、申請用の `POST` APIと、当月1か月分のデータを取得する `GET` APIの2つを追加します。

ここでは初学者にも分かりやすいよう、既存の打刻リポジトリなどを活用して組み立てるロジックの骨子を示します。

```
// AttendanceController.java 等への追加イメージ

/**
 * ユーザー自身の当月1か月分の打刻・申請状況リストを取得する
 * GET /api/attendance/monthly-list?accountId=xxx&yearMonth=2026-05
 */
@GetMapping("/monthly-list")
public ResponseEntity<?> getMonthlyAttendanceList(
    @RequestParam String accountId,
    @RequestParam String yearMonth) {
```

```

// 指定された年月の1日から末日までの打刻レコードをRepositoryから取得して返却します
// 申請状態 (is_timechange_demand) や承認ステータスも一緒にフロントに送ります
return ResponseEntity.ok(/* 該当月のレコードリスト */);
}
/**
 * 打刻内容の編集申請（修正申請）を提出する
 * POST /api/attendance/request-edit
 */
@PostMapping("/request-edit")
public ResponseEntity<String> requestAttendanceEdit(@RequestBody TimeChangeRequest request) {

    // 1. バリデーション：時刻形式チェックや備考の空チェック
    if (request.getReason() == null || request.getReason().trim().isEmpty()) {
        return ResponseEntity.badRequest().body("修正理由（備考）の入力は必須です。");
    }
    try {
        // 2. 該当日のレコードが既に存在するか検索
        // 存在すれば、そのレコードの申請中カラムを書き換え、フラグを「1:申請中」、ステータスを「0:未承認」にする
        // 存在しなければ（まるごと押し忘れた日）、新規にレコードを生成して保存する

        System.out.println("=== [打刻申請] 編集申請を受け付けました。日付: " + request.getRecordDate() + " ===");
        return ResponseEntity.ok("打刻修正申請を提出しました。管理者の承認をお待ちください。");
    } catch (Exception e) {
        return ResponseEntity.status(500).body("申請処理中にエラーが発生しました。");
    }
}

```

フロントエンド（Vue.js）の画面設計（UXイメージ）

フロントエンド側（`AttendanceEditView.vue`）では、仕様書通りに以下のような使い心地の良い2部構成のUIを表現します。

- **左側（または上段）：当月カレンダーリスト**
 - 1日～末日までの行がズラリと並び、「その日の実際の打刻時刻」と「現在の申請ステータス（申請中 / 差戻 / 承認済）」がバッジで一目でわかる一覧表。
 - 各行の右端に「編集申請」ボタンを配置。
- **右側（または下段）：モーダル、あるいは動的ポップアップフォーム**
 - リストのボタンを押すと、その日の日付が自動セットされた入力フォームがフワッと出現。
 - 既に打刻データがある場合は、初期値としてその時刻が最初から入力欄にセットされる親切設計（UX改善）。

次に進むための確認

この「打刻内容編集申請」を進めるにあたり、バックエンドのJPAエンティティに、「**修正案のEntry/Exit時刻**」や「**申請理由（備考テキスト）**」を保存するためのカラム（フィールド）が既に定義されているか、あるいは今回新しく追加する必要があるかによって、Java側のソースコードの書き方が少し変わってきます。

この時点の `AttendanceRecord.java` は「1回ごとの打刻（`clock-in` または `clock-out`）を縦に1レコードずつインサートしていくログ形式」の設計になっているため、今回の仕様書が求める「1か月分のリストカレンダーを1日1行で表示し、Entry/Exit時刻を1つのフォームで修正・申請する」という要件をそのまま実装するには、少し工夫（拡張）が必要な状態です。

現在のエンティティと仕様書の「ギャップ」と解決策

- 仕様書が求めるテーブル構造（`entry_exit_calendar`）

仕様書では、1つのレコードに「その日のEntry（出勤）」と「その日のExit（退勤）」が横並びで1行に格納される構成を想定しています。また、そこに `is_timechange_demand`（修正申請フラグ）や `timechange_status`（承認ステータス）などの管理用フィールドを持たせる指定があります。

- 現在の `AttendanceRecord.java` の構造

現在のエンティティは、1つのログ（行）に `type: "clock-in"` または `"clock-out"` を識別して保存する「縦持ちの履歴ログ形式」になっています（前回の `pairedHistory` でフロント側でペアに復元していたのはこのためです）。

スマートな解決策

現在の打刻ログ形式（`AttendanceRecord.java`）をそのまま壊さずに、仕様書で指定されている

`entry_exit_calendar` テーブルに相当する新しいエンティティ（`EntryExitCalendar.java`）を1つ新設するアプローチが最も安全です。

これを行うことで、

- 日々の打刻はこれまで通り高速にログ保存する
 - 編集申請やカレンダー表示、将来の「予定申請」は、日付（`LocalDate`）を主軸にした専用の `EntryExitCalendar` でスマートに一元管理する
- という、実務のWeb開発でもよく使われる「役割分担（データモデルの分離）」が実現できます。

今回の実装事項に最適化した新エンティティの作成

それでは、仕様書の要件を100%満たし、今回の「打刻内容編集申請」に完璧に対応できる

`EntryExitCalendar.java` を作成しましょう。

`entity` パッケージ配下に、新しく以下のファイルを作成してください。初学者でも直感的に理解できるよう、仕様書通りのフィールド名と詳細なコメントを添えています。

```
package com.appspace.backend.entity;
```

```
import jakarta.persistence.*;
```

```
import lombok.Data;
```

```

import java.time.LocalDate;
@Entity
@Table(name = "entry_exit_calendar") // 仕様書準拠のテーブル名
@Data
public class EntryExitCalendar {

    @Id
    @Column(name = "record_id", length = 50)
    private String recordId; // 登録日の yyyyMMddhhmmssss + UUID末尾4桁

    @Column(name = "record_date", nullable = false)
    private LocalDate recordDate; // カレンダーの「日付」 (yyyy-MM-dd)

    @Column(name = "registered_account_id", nullable = false)
    private String registeredAccountId; // 申請ユーザーの accountId

    // --- 💡 確定・承認済みの実際の打刻時刻 (Stringの hh:mm:ss 形式で扱うとフロントとの連携が劇的に楽になります) ---
    @Column(length = 10)
    private String entryTime; // 確定Entry時刻 (例: "09:00:00")

    @Column(length = 10)
    private String exitTime; // 確定Exit時刻 (例: "18:00:00")

    // --- 💡 編集申請中 (管理者承認待ち) のテンポラリ時刻保存用フィールド ---
    @Column(length = 10)
    private String tmpEntryTime; // 申請中のEntry時刻案

    @Column(length = 10)
    private String tmpExitTime; // 申請中のExit時刻案

    // --- 💡 仕様書に記載された状態管理フラグとメッセージ ---
    @Column(name = "is_schedule_demand")
    private int isScheduleDemand = 0; // 予定申請中フラグ (0:なし, 1:申請中)

    @Column(name = "schedule_status")
    private int scheduleStatus = 0; // 予定申請ステータス (0:未承認, 1:差戻, 2:承認済)

    @Column(name = "is_timechange_demand")
    private int isTimechangeDemand = 0; // 時刻修正申請中フラグ (0:なし, 1:申請中)

    @Column(name = "timechange_status")
    private int timechangeStatus = 0; // 時刻修正ステータス (0:未承認, 1:差戻, 2:承認済)

    @Column(columnDefinition = "TEXT")
    private String reason; // ユーザーからの修正理由・備考

    @Column(columnDefinition = "TEXT")
    private String adminComment; // 管理者からの差戻理由・コメント
}

```

Repository（データアクセス層）の作成

この新しいエンティティを操作するために、JPAリポジトリを `repository` パッケージ配下に作成します。

「ユーザーID」と「指定された年月（1日～末日）」でデータをシュッとまとめて引っ張ってこれるように、クエリメソッドを1つ用意します。

```
package com.appspace.backend.repository;

import com.appspace.backend.entity.EntryExitCalendar;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.time.LocalDate;
import java.util.List;
import java.util.Optional;

@Repository
public interface EntryExitCalendarRepository extends JpaRepository<EntryExitCalendar, String> {
```

```
/**
 * 特定ユーザーの、指定された期間（1か月分など）のレコードを日付順にすべて取得する
 */
List<EntryExitCalendar> findByRegisteredAccountIdAndRecordDateBetweenOrderByRecordDateAsc(
    String accountId, LocalDate startDate, LocalDate endDate);

/**
 * 特定ユーザーの、特定の日付のレコードを1件取得する
 */
Optional<EntryExitCalendar> findByRegisteredAccountIdAndRecordDate(String accountId, LocalDate recordDate);
}
```

ここまでのまとめ

日々の打刻ログを蓄積するだけの構造（AttendanceRecord）から「1か月分の日付が1行ずつ並んだカレンダー表」を表現するには、日付を主軸にした別の帳簿（EntryExitCalendar）が必要だったのです。

その為今回、EntryExitCalendar 用に新しくエンティティとリポジトリを追加したのでした。

ここまでの実装により、ServiceやControllerへの具体的なロジックの肉付けへ進むことができます。

解説：リポジトリ用インターフェースの役割と記述の意味

そもそも「リポジトリ用インターフェース」とは？

一言でいうと、リポジトリは「Javaのプログラム（オブジェクト）と、データベース（SQL）の間を翻訳して繋ぐ、専属のデータ出し入れ窓口」です。

通常、データベースからデータを取得したり保存したりするには、`SELECT * FROM ... WHERE ...` といった SQL（データベース専用の言語）を細かく記述する必要がありますが、SpringBootの「JPA（Java Persistence API）」という仕組みを使うと、このリポジトリのインターフェースを1枚定義するだけで、**SQLを1行も書かなくてもJavaのメソッドを呼び出す感覚でデータの検索・保存・削除が自動的に行える**ようになります。

「インターフェース（中身のない殻）」として定義するだけで、SpringBootが起動時に「よし、この通りに動く本番用の窓口を裏側で自動作成するぞ」と身代わりを作ってくれるため、私たちは非常に楽をすることができます。

今回定義したリポジトリの記述の意味

先ほど作成した `EntryExitCalendarRepository` に書かれているコードを分解して、その「意味」を概ねで解説します。

```
public interface EntryExitCalendarRepository extends JpaRepository<EntryExitCalendar, String> {
```

- `extends JpaRepository<EntryExitCalendar, String>` の意味:
JPAが最初から用意してくれている「超万能な基本セット」を継承（ダウンロード）しています。
これによって、自分で何も書かなくても、全件取得（`findAll()`）、ID検索（`findById()`）、保存（`save()`）、削除（`deleteById()`）といった基本操作が最初から使える状態になります。
- 後ろの `<EntryExitCalendar, String>` は、「`EntryExitCalendar` エンティティを扱い、その主キー（`@Id`）の型は `String`（UUID）ですよ」とJPAに教えてあげるための指定です。


```
List<EntryExitCalendar> findByRegisteredAccountIdAndRecordDateBetweenOrderByRecordDateAsc(  
    String accountId, LocalDate startDate, LocalDate endDate);
```

このメソッド定義が担っている意味

これがJPAの根幹の機能で、「メソッドの名前のルール」に従って命名するだけで、JPAが自動的にSQL文を脳内で組み立ててくれる仕組み（クエリメソッド）です。名前をパーツごとに分解してみると、何をしようとしているのかが分かります。

- `List<EntryExitCalendar>` : 検索結果は、条件に合うレコードが複数行（1か月分）入ったリストで返してね、という意味です。
- `findBy` : 「～を検索してください」という合図（SQLの `SELECT` に相当）。
- `RegisteredAccountId` : 登録ユーザーのID（ `registeredAccountId` ）が一致するものを探してね（SQLの `WHERE registered_account_id = ?` ）。
- `And` : 「かつ」という条件の結合。

- **RecordDateBetween** : 日付 (`recordDate`) が、〇〇から〇〇の「間 (Between)」にあるものを探してね (SQLの `WHERE record_date BETWEEN ? AND ?`)。
- **OrderByRecordDateAsc** : 結果は日付 (`recordDate`) が古い順の「昇順 (Ascending)」で並び替えて並べてね (SQLの `ORDER BY record_date ASC`)。

引数の意味

上記の検索条件の「? (具体的な値)」に放り込むための変数たちです。

「誰の (`accountId`)」「何月1日から (`startDate`)」「何月31日まで (`endDate`)」という具体的な指示書を、この変数を通じてリポジトリ窓口に手渡します。

```
Optional<EntryExitCalendar> findByRegisteredAccountIdAndRecordDate(  
    String accountId, LocalDate recordDate);
```

この定義が担っている意味

これも名前のルール通り、「特定のユーザー（RegisteredAccountId）」の「特定の日付（RecordDate）」のレコードを**ピンポイントで1件だけ**探してね、という命令です。

（SQLの WHERE registered_account_id = ? AND record_date = ? ）

- ユーザーが「5月12日の打刻を修正申請したい！」とボタンを押したときに、「すでに5月12日の行（データ）がDBにあるかどうか」をチェックするために使います。
- Optional<...> という型で囲っている理由は、「まだ打刻していない未来の日付や、丸ごと押し忘れた日」を検索した際、結果が『空っぽ（データが存在しない）』になる可能性があるためです。
 - Javaでエラーを起こさずに「空っぽ」を扱うための保護カプセルだと思ってください。

Service（ビジネスロジック）層の実装

`service` パッケージ配下に、新しく `EntryExitCalendarService.java` を作成します。
ここには、以下の2つの核となる「業務ロジック（仕組み）」を記述します。

- 当月1か月分のカレンダー枠を自動生成してデータを詰め込むロジック
 - データベースにデータがある日はそれを使い、まだ打刻がない未来の日や押し忘れた空の日であっても、Vue側で1か月分の綺麗なリストが表示できるよう、プログラム側で空のカレンダー行を優しく補完してあげる親切設計です。
- 編集申請を受け付け、DBの状態を「申請中」に更新するロジック

```
package com.appspace.backend.service;  
  
import com.appspace.backend.entity.EntryExitCalendar;  
import com.appspace.backend.repository.EntryExitCalendarRepository;  
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.stereotype.Service;
```

```
import org.springframework.transaction.annotation.Transactional;

import java.time.LocalDate;
import java.time.YearMonth;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;
import java.util.UUID;

@Service
public class EntryExitCalendarService {

    @Autowired
    private EntryExitCalendarRepository calendarRepository; // 先ほど作った翻訳窓口（リポジトリ）を接続

    /**
     * 業務ロジック1: 指定された年月の1か月分のリストカレンダーを取得・生成する
     * @param accountId 対象ユーザーのUUID
     * @param yearMonthStr 対象年月 (例: "2026-05")
     */
    public List<EntryExitCalendar> getMonthlyCalendar(String accountId, String yearMonthStr) {
        // 1. "2026-05" という文字列を、Javaが計算できる YearMonth 型に変換します
        YearMonth targetMonth = YearMonth.parse(yearMonthStr);

        // 2. その月の「1日」と「末日」を割り出します
        LocalDate startDate = targetMonth.atDay(1);
        LocalDate endDate = targetMonth.atEndOfMonth();
    }
}
```

```

// 3. リポジトリ窓口を使って、すでにDBに登録されている該当期間のレコードを日付順にシュッと取得します
List<EntryExitCalendar> existingRecords = calendarRepository
    .findByRegisteredAccountIdAndRecordDateBetweenOrderByRecordDateAsc(accountId, startDate, endDate);

// 4. 【ここが職人技】1日から末日までの「完璧な1か月分のリスト」を入れる器を用意します
List<EntryExitCalendar> fullMonthCalendar = new ArrayList<>();

// 5. 1日から末日 まで1日ずつループを回します
for (LocalDate date = startDate; !date.isAfter(endDate); date = date.plusDays(1)) {
    final LocalDate currentLoopDate = date; // ループ内の日付を固定化

    // すでにDBにレコードが存在するかチェックします
    Optional<EntryExitCalendar> foundRecord = existingRecords.stream()
        .filter(r -> r.getRecordDate().equals(currentLoopDate))
        .findFirst();

    if (foundRecord.isPresent()) {
        // すでにデータが存在すれば、それをそのままリストに採用！
        fullMonthCalendar.add(foundRecord.get());
    } else {
        // 💡 まだ打刻データがない日（未出勤の日や未来の日）の場合、
        // 画面で「空行」として表示できるように、その場で見映え用の空レコードを偽造してあげます
        EntryExitCalendar blankDay = new EntryExitCalendar();
        blankDay.setRecordDate(currentLoopDate);
        blankDay.setRegisteredAccountId(accountId);
        blankDay.setEntryTime("-"); // 時刻はまだないのでハイフン
        blankDay.setExitTime("-");
        // 各種フラグやステータスも初期状態（0）のままセット
        fullMonthCalendar.add(blankDay);
    }
}

return fullMonthCalendar; // 1日～末日まで1行の漏れもない完璧な1か月分データを返却！
}

```

```

/**
 * 業務ロジック2: ユーザーからの打刻修正（編集）申請を処理する
 */
@Transactional // データの書き換えを安全に行うための宣言（エラーが起きたら自動で巻き戻す）
public void requestTimeChange(String accountId, LocalDate recordDate, String tmpEntry, String tmpExit, String reason) {

    // 1. まず、その日のレコードが既にDBにあるかどうかをピンポイント検索します
    Optional<EntryExitCalendar> existingOpt = calendarRepository
        .findByRegisteredAccountIdAndRecordDate(accountId, recordDate);

    EntryExitCalendar targetRow;

    if (existingOpt.isPresent()) {
        // すでに打刻データがある日なら、その既存データを書き換え対象にします
        targetRow = existingOpt.get();
    } else {
        // 丸ごと押し忘れてデータが1件もない日の場合、新しく行を立ち上げます
        targetRow = new EntryExitCalendar();

        // 仕様書通りのルール（登録日の yyyyMMddhhmmssss + UUID末尾4桁）で record_id を生成
        String timePart = java.time.LocalDateTime.now().format(java.time.format.DateTimeFormatter.ofPattern("yyyyMMddHHmmssSSS"));
        String uuidPart = UUID.randomUUID().toString().substring(32); // 末尾4桁を取得
        targetRow.setRecordId("REC" + timePart + uuidPart);

        targetRow.setRecordDate(recordDate);
        targetRow.setRegisteredAccountId(accountId);
        targetRow.setEntryTime("-"); // 元の打刻時刻は空なのでハイフン
        targetRow.setExitTime("-");
    }
}

```

```

// 2. 仕様書に指定された「申請中ステータス」の値をパチパチとセットします
targetRow.setTmpEntryTime(tmpEntry); // ユーザーが希望する新しいEntry時刻
targetRow.setTmpExitTime(tmpExit);   // ユーザーが希望する新しいExit時刻
targetRow.setIsTimechangeDemand(1);  // ★時刻修正申請中フラグを「1:申請中」にする
targetRow.setTimechangeStatus(0);    // ★ステータスを「0:未承認」にする
targetRow.setReason(reason);          // 修正の理由（備考）を格納
targetRow.setAdminComment("");        // 申請し直しに備えて管理者の過去コメントをクリア

// 3. 翻訳窓口（リポジトリ）に「これで保存して！」と手渡します
calendarRepository.save(targetRow);
System.out.println("=== [Service] DBへの申請ステータス書き込みが正常完了しました ===");
}
}

```

Controller（窓口・受付）層の実装

続いて、フロントエンド（Vue.js）からのネットワーク通信（HTTPリクエスト）を一番に受け取る受付窓口 `AttendanceEditController.java` を `controller` パッケージ配下に新設します。

ここでは、仕様書のセキュリティ要件である「SpringBoot側での厳格な二重バリデーション（空チェックや時刻形式チェック）」をきっちり組み込んでいます。


```
package com.appspace.backend.controller;

import com.appspace.backend.dto.TimeChangeRequest;
import com.appspace.backend.entity.EntryExitCalendar;
import com.appspace.backend.service.EntryExitCalendarService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/attendance")
@CrossOrigin(origins = "https://*.app.github.dev") // Codespaces環境のCORS対策
public class AttendanceEditController {

    @Autowired
    private EntryExitCalendarService calendarService; // ビジネスロジックの職人を呼び出せるように接続

    /**
     * 窓口1: ユーザー自身の指定年月の1か月分のカレンダーリストを返却する
     * GET http://localhost:8080/api/attendance/monthly-list?accountId=xxx&yearMonth=2026-05
     */
    @GetMapping("/monthly-list")
    public ResponseEntity<?> getMonthlyList(
        @RequestParam String accountId,
        @RequestParam String yearMonth) {
        try {
            // サービス職人に依頼して、完璧な1か月分リストを作ってもらいます
            List<EntryExitCalendar> list = calendarService.getMonthlyCalendar(accountId, yearMonth);
            return ResponseEntity.ok(list);
        } catch (Exception e) {
            return ResponseEntity.internalServerError().body("カレンダーデータの取得に失敗しました: " + e.getMessage());
        }
    }
}
```

```

/**
 * 窓口2: ユーザーからの打刻修正（編集）申請リクエストを受け付ける
 * POST http://localhost:8080/api/attendance/request-edit
 */
@PostMapping("/request-edit")
public ResponseEntity<String> requestEdit(@RequestBody TimeChangeRequest request) {

    // 【二重バリデーション】仕様書要件に基づく厳格なバックエンドチェック

    // 1. 備考（修正理由）の空っぽチェック
    if (request.getReason() == null || request.getReason().trim().isEmpty()) {
        return ResponseEntity.badRequest().body("【却下】修正理由（備考）の入力は必須です。");
    }

    // 2. 時刻が正しい形式（hh:mm:ss）になっているか正規表現を使ってチェック
    // 「数字2桁：数字2桁」の形、または未打刻を埋めるハイフン「-」のみ許容
    String timeRegex = "^[01][0-9]|2[0-3]):[0-5][0-9]:[0-5][0-9]$";

    if (!request.getTargetEntryTime().equals("-") && !request.getTargetEntryTime().matches(timeRegex)) {
        return ResponseEntity.badRequest().body("【却下】Entry時刻の形式が不正です。(hh:mm:ssで入力してください)");
    }
    if (!request.getTargetExitTime().equals("-") && !request.getTargetExitTime().matches(timeRegex)) {
        return ResponseEntity.badRequest().body("【却下】Exit時刻の形式が不正です。(hh:mm:ssで入力してください)");
    }

    try {
        // すべてのチェックをクリアしたら、サービス層にDBへの書き込みを命じます
        calendarService.requestTimeChange(
            request.getAccountId(),
            request.getRecordDate(),
            request.getTargetEntryTime(),
            request.getTargetExitTime(),
            request.getReason()
        );

        System.out.println("=== [Controller] 修正申請の受付に成功しました ===");
        return ResponseEntity.ok("打刻内容の修正申請を提出しました。管理者の承認をお待ちください。");
    } catch (Exception e) {
        return ResponseEntity.internalServerError().body("申請処理中に重大なエラーが発生しました: " + e.getMessage());
    }
}

```

SecurityConfig.java へのURL追加確認

新しく作った `/api/attendance/monthly-list` や `/api/attendance/request-edit` にフロントからアクセスできるように、以前行ったのと同様に `SecurityConfig.java` の許可リスト（`.requestMatchers(...)`）の中に以下の一行を優しく追加してあげてください。

```
"/api/attendance/**", // ★打刻および申請に関連するパスを一括許可リストに追加
```

バックエンドのビルド & 起動テスト

ソースコードの記述が完了したら、SpringBootを再起動（ビルド）してみましょう！

エラーなく起動すれば、「打刻内容編集申請」を受け付ける裏側のすべての防壁とパイプラインが完全に開通した状態になります。

フロントエンド側の実装

「操作日の当月1か月分のリストカレンダーを表示し、行を押すと編集フォームが出現する」というUXを満たすため、 `AttendanceEditView.vue` を新設して盛り込んでいきましょう。

`AttendanceEditView.vue` の実装

```
<template>
  <div class="attendance-edit-container">
    <h2>打刻内容編集申請</h2>
    <p class="subtitle">当月1か月分の打刻履歴の確認と、修正申請が行えます。</p>

    <div class="month-selector">
      <button @click="changeMonth(-1)" class="btn-arrow">◀ 前月</button>
      <span class="current-month-text">{{ currentYearMonth }}</span>
      <button @click="changeMonth(1)" class="btn-arrow">次月 ▶</button>
    </div>

    <div class="calendar-list-wrapper">
      <table class="calendar-table">
```

```

<thead>
  <tr>
    <th>日付</th>
    <th>確定 Entry</th>
    <th>確定 Exit</th>
    <th>申請状態</th>
    <th>操作</th>
  </tr>
</thead>

<tbody>
  <tr v-for="day in calendarList" :key="day.recordDate" :class="getRowClass(day.recordDate)">
    <td class="date-col">{{ formatDate(day.recordDate) }}</td>

    <td><span class="time-text">{{ day.entryTime }}</span></td>
    <td><span class="time-text">{{ day.exitTime }}</span></td>

    <td>
      <span v-if="day.isTimechangeDemand === 1" class="status-badge" :class="getStatusClass(day.timechangeStatus)">
        {{ getStatusLabel(day.timechangeStatus) }}
      </span>
      <span v-else class="status-none">-</span>
    </td>

    <td>
      <button @click="openEditForm(day)" class="btn-action-edit">
        {{ day.isTimechangeDemand === 1 ? '再申請' : '編集申請' }}
      </button>
    </td>
  </tr>
</tbody>

```



```
<div class="form-group">
  <label>備考・修正理由 <span class="required">※必須</span></label>
  <textarea
    v-model="form.reason"
    placeholder="例：出勤時に打刻を失念したため。〇〇クライアント直行のため。"
    rows="3"
  ></textarea>
</div>

<div v-if="selectedDay.timechangeStatus === 1 && selectedDay.adminComment" class="admin-comment-box">
  <strong>管理者からの差戻理由:</strong>
  <p>{{ selectedDay.adminComment }}</p>
</div>
</div>

<div class="form-actions">
  <button @click="submitRequest" class="btn-submit">申請を提出する</button>
  <button @click="closeForm" class="btn-cancel">キャンセル</button>
</div>
</div>
</div>

</div>
</template>
```

```
<script setup>
import { ref, onMounted } from 'vue';
import apiClient from '../api';

// 状態管理変数
const calendarList = ref([]);
const currentYearMonth = ref(''); // "2026-05" 形式で保持
const loggedInAccountId = ref('');

// フォーム用のリアクティブ変数
const showForm = ref(false);
const selectedDay = ref(null);
const form = ref({
  entryTime: '',
  exitTime: '',
  reason: ''
});

// 初期設定（現在の年月をセット）
const initCurrentMonth = () => {
  const now = new Date();
  const year = now.getFullYear();
  const month = String(now.getMonth() + 1).padStart(2, '0');
  currentYearMonth.value = `${year}-${month}`;
};
```



```
// 1か月分のカレンダーリストをバックエンドから取得する関数
const fetchMonthlyData = async () => {
  if (!loggedInAccountId.value) return;
  try {
    const response = await apiClient.get('/attendance/monthly-list', {
      params: {
        accountId: loggedInAccountId.value,
        yearMonth: currentYearMonth.value
      }
    });
    calendarList.value = response.data;
  } catch (error) {
    console.error('カレンダーデータの取得に失敗しました:', error);
  }
};

// 月の切り替え（前月 / 次月）
const changeMonth = (offset) => {
  const [year, month] = currentYearMonth.value.split('-').map(Number);
  const date = new Date(year, month - 1 + offset, 1);
  const nextYear = date.getFullYear();
  const nextMonth = String(date.getMonth() + 1).padStart(2, '0');
  currentYearMonth.value = `${nextYear}-${nextMonth}`;
  fetchMonthlyData(); // 月が変わったらデータを再読み込み
};
```

```
// 編集フォームを開く処理（既存の打刻内容を初期値として反映する親切UX設計）
const openEditForm = (day) => {
  selectedDay.value = day;

  // 既に申請中のデータがあればそれを、なければ確定時刻を、それもない場合は空欄（または現在の時刻等）をセット
  form.value.entryTime = day.isTimechangeDemand === 1 ? day.tmpEntryTime : (day.entryTime === '-' ? '09:00:00' : day.entryTime);
  form.value.exitTime = day.isTimechangeDemand === 1 ? day.tmpExitTime : (day.exitTime === '-' ? '18:00:00' : day.exitTime);
  form.value.reason = day.reason || '';

  showForm.value = true;
};

const closeForm = () => {
  showForm.value = false;
  selectedDay.value = null;
};

// 仕様書要件：フロントエンド側での厳格なバリデーション&確認アラート
const submitRequest = async () => {
  // 1. 備考欄の空チェック
  if (!form.value.reason.trim()) {
    alert('修正理由（備考）を入力してください。');
    return;
  }

  // 2. 時刻形式チェック (hh:mm:ss)
  const timeRegex = "^[01][0-9]|2[0-3]:[0-5][0-9]:[0-5][0-9]$";
  if (!form.value.entryTime.match(timeRegex) || !form.value.exitTime.match(timeRegex)) {
    alert('時刻は「hh:mm:ss」の形式（例: 09:00:00）で入力してください。');
    return;
  }
}
```

```
// 3. 仕様書要件：申請時は必ず確認アラートを表示する
if (!confirm('この内容で打刻内容の編集申請を提出してもよろしいですか？')) {
  return;
}

try {
  // バックエンドの POST /api/attendance/request-edit へ送信
  await apiClient.post('/attendance/request-edit', {
    accountId: loggedInAccountId.value,
    recordDate: selectedDay.value.recordDate,
    targetEntryTime: form.value.entryTime,
    targetExitTime: form.value.exitTime,
    reason: form.value.reason
  });

  alert('申請が正常に提出されました！');
  closeForm();
  fetchMonthlyData(); // リストを最新状態に更新
} catch (error) {
  console.error('申請エラー:', error);
  alert(error.response?.data || '申請の提出に失敗しました。');
}
};
```

```
// 補助・装飾用ユーティリティ関数群
const formatDate = (dateStr) => {
  if (!dateStr) return '';
  const [y, m, d] = dateStr.split('-');
  return `${Number(m)}月${Number(d)}日`;
};

// 仕様書通りのステータスコードをテキストに変換
const getStatusLabel = (status) => {
  if (status === 0) return '申請中 (未承認)';
  if (status === 1) return '差戻し';
  if (status === 2) return '承認済み';
  return '不明';
};

const getStatusClass = (status) => {
  if (status === 0) return 'badge-pending';
  if (status === 1) return 'badge-rejected';
  if (status === 2) return 'badge-approved';
  return '';
};

const getRowClass = (dateStr) => {
  if (!dateStr) return '';
  const dayNum = new Date(dateStr).getDay();
  if (dayNum === 6) return 'row-saturday';
  if (dayNum === 0) return 'row-sunday';
  return '';
};
```

```
// 画面展開時の初期ロード
onMounted(() => {
  initCurrentMonth();
  const userData = localStorage.getItem('user');
  if (userData) {
    const user = JSON.parse(userData);
    loggedInAccountId.value = user.accountId;
    fetchMonthlyData(); // データ取得開始
  } else {
    alert('ユーザー情報が取得できません。再ログインしてください。');
  }
});
</script>

<style scoped>
.attendance-edit-container {
  max-width: 900px;
  margin: 40px auto;
  padding: 20px;
  font-family: sans-serif;
  color: #333;
}
.subtitle { color: #666; margin-bottom: 30px; }

/* 月選択のデザイン */
.month-selector {
  display: flex;
  justify-content: center;
  align-items: center;
  gap: 20px;
  margin-bottom: 20px;
  background-color: #343a40; padding: 10px; border-radius: 6px;
}
.btn-arrow {
  background-color: #6c757d;
  border: 1px solid #5a6268;
  color: #ffffff;
  padding: 8px 16px;
  border-radius: 4px;
  cursor: pointer;
  font-weight: bold;
  transition: background-color 0.2s, border-color 0.2s; /* ホバー時の変化を滑らかに */
}
```

```
.btn-arrow:hover {
  background-color: #5a6268;
  border-color: #4e555b;
}
.current-month-text {
  font-size: 1.3rem;
  font-weight: bold;
  min-width: 120px;
  text-align: center;
  color: #e0e0e0;
  text-shadow: 0 1px 2px rgba(0, 0, 0, 0.2);
}

/* テーブルカレンダーのデザイン */
.calendar-list-wrapper {
  background: white;
  border-radius: 8px;
  box-shadow: 0 4px 12px rgba(0,0,0,0.08);
  overflow: hidden;
}
.calendar-table { width: 100%; border-collapse: collapse; text-align: center; }
.calendar-table th {
  background-color: #f8f9fa;
  color: #495057;
  padding: 14px;
  font-weight: 600;
  border-bottom: 2px solid #dee2e6;
}
.calendar-table td { padding: 12px; border-bottom: 1px solid #e0e0e0; vertical-align: middle; }
```

```
/* 土日の行に優しい背景色を塗る安全策 */
.row-saturday { background-color: #f7faff; }
.row-sunday { background-color: #fff7f7; }
.date-col { font-weight: bold; color: #455a64; }
.time-text { font-family: monospace; font-size: 1rem; }

/* ステータスバッジの装飾 */
.status-badge { display: inline-block; padding: 4px 10px; border-radius: 12px; font-size: 0.85rem; font-weight: bold; }
.badge-pending { background-color: #fff3e0; color: #f57c00; } /* 申請中：薄いオレンジ */
.badge-rejected { background-color: #ffebee; color: #d32f2f; } /* 差戻：薄い赤 */
.badge-approved { background-color: #e8f5e9; color: #388e3c; } /* 承認：薄い緑 */
.status-none { color: #ccc; }

.btn-action-edit {
  background-color: #ffffff;
  border: 1px solid #007bff;
  color: #007bff;
  padding: 6px 12px;
  border-radius: 4px;
  cursor: pointer;
  transition: all 0.2s;
}
.btn-action-edit:hover { background-color: #007bff; color: white; }
```

```
/* 編集申請ポップアップフォームの装飾 */
.edit-form-overlay {
  position: fixed;
  top: 0; left: 0;
  width: 100%;
  height: 100%;
  background: rgba(0,0,0,0.4);
  display: flex;
  justify-content: center;
  align-items: center;
  z-index: 1000;
}
.edit-form-card {
  background: white;
  padding: 30px;
  border-radius: 8px;
  width: 100%;
  max-width: 450px;
  box-shadow: 0 10px 25px rgba(0,0,0,0.15);
  animation: fadeIn 0.2s ease-out;
}
```



```
@keyframes fadeIn {  
  from { transform: translateY(10px); opacity: 0; }  
  to { transform: translateY(0); opacity: 1; }  
}  
  
.form-body { margin: 20px 0; text-align: left; }  
.form-group { margin-bottom: 18px; }  
.form-group label {  
  display: block;  
  margin-bottom: 6px;  
  font-weight: bold;  
  font-size: 0.9rem;  
  color: #555;  
}  
.form-group input[type="text"], .form-group textarea {  
  width: 100%;  
  padding: 10px;  
  border: 1px solid #ccc;  
  border-radius: 4px;  
  box-sizing: border-box;  
  font-size: 1rem;  
}  
.form-group input[type="text"] { font-family: monospace; }  
.required { color: #d32f2f; }
```

```
.admin-comment-box {
  background-color: #fff3r0;
  border-left: 4px solid #f57c00;
  padding: 10px;
  margin-top: 15px;
  border-radius: 4px;
  font-size: 0.9rem;
  background: #fff8f0;
}

.form-actions { display: flex; gap: 10px; margin-top: 25px; }
.btn-submit {
  flex: 1;
  background-color: #4caf50;
  color: white;
  border: none;
  padding: 12px;
  border-radius: 4px;
  font-size: 1rem;
  font-weight: bold;
  cursor: pointer;
}
.btn-submit:hover { background-color: #43a047; }
.btn-cancel {
  background-color: #e0e0e0;
  color: #333;
  border: none;
  padding: 12px 20px;
  border-radius: 4px;
  font-size: 1rem;
  cursor: pointer;
}
.btn-cancel:hover { background-color: #d5d5d5; }
</style>
```

router/index.js (Vue Router) への追加

この画面にダッシュボード等からジャンプできるよう、ルーティング（URLパス）の登録を行いましょう。

router/index.js の routes 配列の中に、以下のように組み込んでください。

```
{
  path: '/attendance-edit',
  name: 'AttendanceEdit',
  component: () => import('../views/AttendanceEditView.vue'),
  // もしNavigation Guard（ログイン必須チェック）を盛り込んでいる場合は、metaタグ等を追加してください
}
```

これで、ダッシュボード画面の下段にある「打刻内容編集申請」ボタンをクリックした際の遷移先が綺麗に繋がります。

連動の確認テスト方法

- **画面の確認:**

「打刻内容編集申請」画面を開くと、今月の1日から末日までの日付が自動生成されて並びます。

- **編集申請のテスト:**

任意の日の「編集申請」ボタンを押すと、右側（中央）に入力ダイアログがフワッと出現します。

- **バリデーションのテスト:**

「備考」を空欄のままにしたり、時刻の秒数を抜いて `09:00` のように入力して申請ボタンを押すと、フロントエンド側のバリデーションが働き、適切な警告メッセージが出るか確かめます。

- **送信確認:**

正しい値（例: `09:15:00` / `18:30:00` / 理由: `打刻押し忘れ`）を入力して送信すると、「**この内容で打刻内容の編集申請を提出してもよろしいですか？**」という確認ダイアログが表示されます。

- **DB連動:**

OKを押した後、画面のカレンダーリストが自動でリロードされ、該当日の右端に「申請中 (未承認)」というオレンジ色の綺麗なバッジがパッと点灯すれば**フロント・バックエンドの完全連動が大成功**です！